

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



SUBJECT: SOFTWARE ARCHITECTURE
PROJECT FINAL REPORT: MOVIE TICKET BOOKING SYSTEM

Instructor : Vu Quang Dung
Group 8: Bui Minh Duc (23010513)
Pham Xuan Bach (23010118)
Nguyen Gia Bao (23010383)
Vu Duc Hieu (23010027)
Course class: CSE703110-1-2-25(N01)

Ha Noi, February 2026

Contents

1. EXECUTIVE SUMMARY	5
1.1 Goal:	5
1.2 Final Achievement:.....	5
2. PROJECT REQUIREMENTS & GOALS	5
2.1 Functional Requirements (Core Features).....	5
2.2 Quality Attributes (Targeted Goals).....	5
3. ARCHITECTURAL DESIGN & IMPLEMENTATION	6
3.1 Technical Stack.....	6
3.2 Implementation of Core Logic (Operational Phases).....	6
4. USE CASE SPECIFICATIONS.....	7
4.1 System & Authentication (UC-01 to UC-03).....	7
4.2 Movie & Showtime Management (Admin) (UC-04 to UC-09)	10
4.3 Browsing & Booking (User) (UC-10 to UC-15).....	15
4.4 Payment (UC-16).....	19
5. DETAILED OPERATIONAL MECHANISMS, INTERACTIONS, AND DATA FLOW	20
5.1 Communication Architecture	20
5.2 Routing Mechanism and API Gateway Pattern.....	21
5.3 Detailed Workflows Analysis.....	21
5.4 Distributed Data Management.....	23
5.5 Fault Tolerance	23
6. TESTING & VERIFICATION	24
6.1 Verification of Requirements (Labs Progress)	24
7. CONCLUSION & REFLECTION	24
7.1 Summary of Lessons Learned	24

Work Assignment Table

Member	Detailed Tasks	Completion Rate
Nguyen Gia Bao	1. Movie Service: Implemented CRUD operations for Movies and Showtimes; handled Cascade Delete logic. 2. Booking Service: Developed core booking logic, atomic transactions for seat locking, and OrderPlaced event publishing. 3. Database Design: Designed and optimized SQLite schemas for movies.db and booking.db. 4. Testing: Conducted Fault Tolerance and resilience testing.	100%
Pham Xuan Bach	1. Infrastructure: Created Dockerfile, docker-compose.yml, and configured the RabbitMQ network. 2. Notification Service: Implemented the Consumer worker to handle email sending and invoice generation events. 3. Payment Service: Developed the Payment REST API and INVOICE_PRINT event publishing.	100%
Bui Minh Duc	1. System Architecture: Designed the C4 Model and defined the Tech Stack. 2. Gateway Service: Developed the API Gateway (app.py), configured request routing and security headers. 3. User Service: Implemented Authentication (Login/Register), Token Verification (UUID), and User Management. 4. Documentation: Wrote the Executive Summary, System Use Cases, and consolidated the Final Report.	100%
Vu Duc Hieu	1. UI/UX Design: Built a responsive SPA frontend using Vanilla JS (ES6+), Bootstrap 5, and Vite; implemented component-based structure, state handling, and client-side routing. 2. Client Integration: Developed a reusable API client layer (myClient.js) handling request abstraction, error handling, and token-based authentication with the Gateway.	100%

	3. Admin Pages: Implemented admin dashboard modules (Movies, Showtimes, Analytics) with CRUD flows, form validation, and dynamic table rendering. 4. User Pages: Built the Homepage, Movie Details, Booking flow, and Profile pages.	
--	--	--

1. EXECUTIVE SUMMARY

1.1 Goal:

The primary objective of this project was to refactor a legacy Layered Monolith movie ticket booking application into a distributed Microservices Architecture.

Architectural Pattern: The system adopts a Microservices style utilizing Database-per-Service isolation and a Hybrid Communication Pattern (Synchronous REST APIs for reads and Asynchronous AMQP via RabbitMQ for writes).

1.2 Final Achievement:

The team successfully decomposed the application into independent containers (Gateway, Movie, Booking, Payment, Notification). The final system is orchestrated via Docker Compose, demonstrating high scalability, fault tolerance, and a clear separation of concerns, successfully addressing the limitations of the previous monolithic version.

2. PROJECT REQUIREMENTS & GOALS

2.1 Functional Requirements (Core Features)

- **Movie Management (CRUD):** Admin capabilities to Create, Read, Update, and Delete movies and showtimes.
- **Booking Transaction:** A complex workflow allowing users to select seats, lock them, and process payments.
- **Real-Time Messaging:** Event-driven notifications (Email/Tickets) and Invoice generation upon successful booking.
- **Search & Discovery:** High-performance movie searching for end-users.

2.2 Quality Attributes (Targeted Goals)

- **Scalability:** The ability to scale specific services (e.g., Booking Service) independently during high-traffic events.
- **Availability & Fault Tolerance:** The system must continue to accept bookings even if the Notification or Payment services are temporarily down (achieved via Message Queues).
- **Performance:**
 - *Read Performance:* Fast response times for movie listings using direct API calls.

- *Write Performance:* Non-blocking booking experience using asynchronous processing.

3. ARCHITECTURAL DESIGN & IMPLEMENTATION

3.1 Technical Stack

- Backend: Python 3.13 (Flask) for lightweight microservices.
- Frontend: Vanilla JavaScript (ES6+) and Bootstrap 5, bundled with Vite.
- Database: SQLite (Strict Database-per-Service implementation).
- Message Broker: RabbitMQ (Pika client) for event-driven communication.
- Infrastructure: Docker.

3.2 Implementation of Core Logic (Operational Phases)

Phase A: Infrastructure Bootstrap (Containerization)

- Orchestration: We use `docker-compose.yml` as the infrastructure blueprint. An internal Bridge Network is established to allow services to communicate via Hostnames (Service Discovery) rather than static IPs.
- Idempotent Initialization: Upon startup, each service checks for its specific database file (`movies.db`, `booking.db`). If absent, it automatically executes SQL schemas to create tables and seed initial data. This ensures strict data isolation with no cross-service foreign keys.

Phase B: Synchronous Operations (Read-Heavy Flows)

- API Gateway Pattern: The Flask Gateway (Port 5000) acts as the Single Entry Point. It intercepts Frontend requests, validates UUID-based Security Tokens stored in the database, and routes traffic to internal services (e.g., routing `/api/movies` to `movie-service:5001`).
- Direct Querying: For information retrieval, the Gateway forwards requests to the Movie Service, which queries its local DB and returns JSON directly. This minimizes latency for UI rendering.

Phase C: Asynchronous Operations (Write-Heavy Flows)

- Event-Driven Booking:
 1. Atomic Transaction: The Booking Service locks the seat in its local DB.
 2. Producer: Immediately upon success, it publishes an *OrderPlacedEvent* directly to the *ticket_events* queue.

3. Non-Blocking Response: The user receives a "Success" HTTP 201 response immediately, without waiting for email processing.
- Consumer Workers: The Notification and Payment services run as background workers. They consume messages from the queue to generate invoices and send emails via SMTP. If a worker crashes, the message remains in the queue (Persistence), ensuring no data loss.

Phase D: Technical Optimization

- Frontend Debounce: To prevent API overload, we implemented a custom useDebounce hook in React. This limits search requests by only firing after the user stops typing for 500ms, reducing Gateway load by approximately 80%.

4. USE CASE SPECIFICATIONS

4.1 System & Authentication (UC-01 to UC-03)

UC-01: API Gateway Routing & Verification: Handles request routing, validates API Keys, and verifies User Tokens.

ID and Name:	UC-01: API Gateway Routing & Verification
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	The Gateway acts as the single entry point (Port 5000). It intercepts all requests, validates the API Key, verifies User Tokens (if present), and routes the request to the appropriate microservice.
Trigger:	Any HTTP Request sent to <u>http://localhost:5000</u>
Preconditions:	PRE-1: Gateway Service is running on Port 5000. PRE-2: User Service is running to verify tokens.
Postconditions:	POS-1: Request is successfully forwarded to the downstream service. POS-2: User identity headers (X-User-Email, X-User-Role) are injected.
Normal flow:	1. Client sends a request with Header peko-key. 2. Gateway validates peko-key. 3. Gateway checks for Authorization: Bearer <token> header.

	4. Gateway calls User Service to check token existence in <i>tokens</i> table. 5. User Service returns user profile and role. 6. Gateway injects user info into headers. 7. Gateway forwards request to the target service (Movie/Booking/Payment). 8. Gateway returns the response from the target service to the client.
Alternative Flows:	4a. Invalid Token: If User Service returns 401, Gateway denies request immediately.
Exceptions:	2a. Invalid API Key: Returns 401 Unauthorized. 5a. User Service Down: Returns 500 Internal Server Error.
Priority:	High

UC-02: User Registration: Allows new users to create an account.

ID and Name:	UC-02: User Registration
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	Allows a new user to create an account in the system.
Trigger:	Guest sends: <i>POST /api/auth/register</i>
Preconditions:	PRE-1: Email must not already exist in users.db.
Postconditions:	POS-1: New user record created with role 'user'.
CÒMIRNormal flow:	1. User submits email and password. 2. Gateway forwards to User Service. 3. User Service hashes the password. 4. User Service saves the record to users.db. 5. System returns success message.

Alternative Flows:	none
Exceptions:	4a. Email Exists: Returns 400 Bad Requests.
Priority:	High

UC-03: User Login: Authenticates users and issues Access Tokens (JWT).

ID and Name:	UC-03: User Login
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	Authenticates a user and issues a <i>Database-backed UUID Token</i> for accessingprotected resources.
Trigger:	User sends <i>POST /api/auth/login</i>
Preconditions:	PRE-1: User/admin account must exist.
Postconditions:	POS-1: System returns a valid Access Token. POS-2: Client stores this token for future requests.
Normal flow:	1. User submits email and password. 2. Gateway forwards request to User Service. 3. User Service verifies email exists. 4. User Service validates password hash. 5. User Service generates a random <i>UUID Token</i>, stores it in the <i>tokens</i> table, and returns it. 6. System returns the Token to the user.
SAAlternative Flows:	none
Exceptions:	3a. User Not Found / Wrong Password: Returns 401 Unauthorized.
Priority:	High

4.2 Movie & Showtime Management (Admin) (UC-04 to UC-09)

UC-04: Create Movie: Adds a new movie to the system.

ID and Name:	UC-04: Create Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Adds a new movie title to the system database.
Trigger:	Admin sends POST /api/movies
Preconditions:	PRE-1: User has 'admin' role. PRE-2: Movie ID must be unique.
Postconditions:	POS-1: New movie record is inserted into movies.db.
Normal flow:	1. Admin submits movie details (ID, Title, Genre, Duration). 2. Gateway checks Admin Token. 3. Movie Service checks if ID already exists. 4. Movie Service inserts data into movies table. 5. System returns "Movie Created" message.
Alternative Flows:	none
Exceptions:	3a. Duplicate ID: Returns 400 Bad Request.
Priority:	High

UC-05: Update Movie: Updates existing movie information.

ID and Name:	UC-05: Update Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin

Description:	Allows Admin to correct or update movie information (Title, Genre, Duration).
Trigger:	Admin sends PUT /api/movies/{id}
Preconditions:	PRE-1: Movie exists. PRE-2: User has 'admin' role.
Postconditions:	POS-1: Movie details updated in movies.db.
Normal flow:	1. Admin submits new details (e.g., fix typo in Title). 2. Gateway verifies Admin Token. 3. Movie Service updates the record in database. 4. System returns success message.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-06: Cascade Delete Movie: Deletes a movie along with its showtimes and triggers automatic ticket refunds.

ID and Name:	UC-06: Cascade Delete Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Deletes a movie and automatically deletes all associated showtimes and refunds all booked tickets.
Trigger:	Admin sends DELETE /api/movies/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist.
Postconditions:	POS-1: Movie, Showtimes, and Bookings are deleted.

	POS-2: REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin requests deletion. 2. Movie Service finds all associated showtimes. 3. Loop: Trigger Refund Process for each showtime. 4. Delete all showtimes. 5. Delete the movie record. 6. Returns the count of refunded tickets.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-07: Create Showtime: Schedules a movie and automatically generates available seats.

ID and Name:	UC-07: Create Showtime
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Schedule a movie and generate seats.
Trigger:	Admin sends POST /api/showtimes
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist. PRE-3: Time is valid. PRE-4: The total number of seats must not exceed 64.
Postconditions:	POS-1: Showtime saved in movies.db. POS-2: Seats created in booking.db.

Normal flow:	1. Admin sends time and price. 2. Movie Service saves showtime. 3. Movie Service calls Booking Service (/internal/seats) to generate seats. 4. Returns success.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-08: Update Showtime & Auto-Refund: Updates showtime details (triggers refunds if the price changes).

ID and Name:	UC-08: Update Showtime & Auto-Refund
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Update showtime. If price changes, existing bookings are automatically cancelled and refunded.
Trigger:	Admin sends PUT /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Update successful. POS-2: (If price changed) Tickets cancelled, REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin changes price. 2. Movie Service detects price change.

	3. Calls Booking Service to cancel all tickets. 4. Publishes REFUND_NOTIFICATION to RabbitMQ. 5. Updates new price in DB.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-09: Delete Showtime: Deletes a specific showtime and refunds any sold tickets.

ID and Name:	UC-09: Delete Showtime
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Delete a specific showtime and refund sold tickets.
Trigger:	Admin sends DELETE /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Showtime deleted. POS-2: REFUND_NOTIFICATION is snotification_events queue.
Normal flow:	1. Admin requests deletion. 2. Gateway verifies Admin Role. 3. Trigger Refund Process. 4. Delete record in DB. 5. Returns success message.

Alternative Flows:	none
Exceptions:	none
Priority:	High

4.3 Browsing & Booking (User) (UC-10 to UC-15)

UC-10: Search Movies: Lists all movies or searches for specific ones by title/ID.

ID and Name:	UC-10: Search Movies
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	List all movies or search by title or show one movies by id
Trigger:	GET/api/movies(with?query=...) GET/api/movies/{id}
Preconditions:	PRE-1: Movie Service is running. PRE-2: Database has movie records (optional).
Postconditions:	POS-1: Returns list of movies matching query. POS-2: No database modification.
Normal flow:	1. User sends query. 2. Service performs LIKE search. 3. Returns list.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-11: View Showtimes Schedule: Displays available showtimes and pricing.

ID and Name:	UC-11: View Showtimes Schedule
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	View list of available showtimes and prices.
Trigger:	GET /api/showtimes
Preconditions:	PRE-1: Movie Service is running.
Postconditions:	POS-1: Returns list of showtimes. POS-2: No database modification.
Normal flow:	1. User requests schedule. 2. System returns list of showtimes.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-12: Book Ticket: Reserves a seat and publishes an order event to the system.

ID and Name:	UC-12: Book Ticket
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	User reserves a seat. Price is fetched dynamically.
Trigger:	POST /api/bookings
Preconditions:	PRE-1: User must be logged in. PRE-2: Seat must be 'available'.
Postconditions:	POS-1: Booking created.

	POS-2: The OrderPlacedEvent event is sent to the ticket_events queue.
Normal flow:	1. User selects seat. 2. Service checks availability. 3. Calls Movie Service to fetch price. 4. Creates booking and updates seat status. 5. Sends event to RabbitMQ.
Alternative Flows:	none
Exceptions:	2a. Seat Taken: Returns 400 Bad Request.
Priority:	High

UC-13: View Bookings: Allows users to view their personal booking history.

ID and Name:	UC-13: View Bookings
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	View history of bookings made by the user.
Trigger:	GET /api/bookings
Preconditions:	PRE-1: User must be logged in.
Postconditions:	POS-1: Returns list of bookings for that user.
Normal flow:	1. Gateway extracts email from Token. 2. Service filters bookings by email. 3. Returns list.
Alternative Flows:	none
Exceptions:	none

Priority:	High
-----------	------

UC-14: View Booking Detail: Shows specific details for a single ticket.

ID and Name:	UC-14: View Booking Detail
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	View specific details of a single ticket.
Trigger:	GET /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in. PRE-2: User must own the ticket.
Postconditions:	POS-1: Returns ticket details.
Normal flow:	1. User requests ID. 2. System verifies ownership. 3. Returns details.
Alternative Flows:	none
Exceptions:	2a. Not Owner: Returns 403 Forbidden.
Priority:	High

UC-15: Cancel Booking (Manual): Allows users to manually cancel their own bookings.

ID and Name:	UC-15: Cancel Booking (Manual)
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	User cancels their own ticket. Note: Manual cancellation does NOT trigger a refund event in the current implementation.
Trigger:	DELETE /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in.

	PRE-2: User must own the ticket.
Postconditions:	POS-1: Booking deleted. POS-2: Seat becomes available.
Normal flow:	1. User requests cancellation. 2. System verifies ownership. 3. Deletes booking. 4. Frees up seat.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

4.4 Payment (UC-16)

UC-16: Process Payment: Processes mock transactions, logs payments, and generates invoices.

ID and Name:	UC-16: Process Payment
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	Pay for a ticket and generate an invoice.
Trigger:	POST /api/payments
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: Valid Booking ID exists.
Postconditions:	POS-1: Payment logged. POS-2: INVOICE_PRINT event sent.

Normal flow:	1. User submits Booking ID. 2. Gateway verifies Token and routes to Payment Service. 3. Payment Service calls Booking Service (Internal API) to verify the booking exists and get the correct amount. 4. Payment Service processes the mock transaction. 5. Payment Service publishes InvoiceGenerated event to RabbitMQ. 6. System returns "Payment Successful" message to User.
Alternative Flows:	none
Exceptions:	3a. Booking Not Found: Payment Service returns 404. 3b. Already Paid: Payment Service returns 400 Bad Request.
Priority:	High

5. DETAILED OPERATIONAL MECHANISMS, INTERACTIONS, AND DATA FLOW

The system is designed based on a distributed Microservices architecture, where components do not communicate via internal function calls (in-process calls) but through network calls. This architecture demands strict regulation of communication protocols and data management.

5.1 Communication Architecture

The system employs a Hybrid Communication Pattern to leverage the advantages of both the synchronous and asynchronous worlds:

- Synchronous Communication (Request/Response):
 - Protocol: HTTP/1.1 RESTful API.
 - Scope: Tasks requiring immediate feedback to be displayed on the User Interface (UI).

- Example: When a user accesses the homepage, the Frontend needs to display the movie list immediately. Using an asynchronous mechanism here would cause unnecessary latency and degrade the User Experience (UX).
- Asynchronous Communication (Event-Driven):
 - Protocol: AMQP (Advanced Message Queuing Protocol) via RabbitMQ.
 - Scope: Complex business tasks, time-consuming processing, or side-effects that do not directly affect the main response.
 - Example: Sending ticket emails and generating invoices are consequences of a "Successful Booking". The user does not need to wait for the email to be sent to know that they have finished booking.

5.2 Routing Mechanism and API Gateway Pattern

The API Gateway (running on port 5000) acts as the Single Entry Point for the entire system.

- Reverse Proxy Mechanism: The Gateway intercepts requests from the Client, analyzes the URL Path, and routes them to the corresponding service container within the internal Docker network.
 - Request GET /api/movies -> Forwarded to movie-service:5001.
 - Request POST /api/bookings -> Forwarded to booking-service:5002.
- Architectural Benefits:
 - Abstraction: The Client does not need to know the internal network structure or the number of services behind the scenes.
 - Security: The Gateway acts as a firewall, blocking invalid requests before they reach the business services.

5.3 Detailed Workflows Analysis

A. Data Query Flow (Read-Heavy Path)

This process focuses on Read Throughput.

1. Initiation: The Frontend calls the API via the Gateway.
2. Service Resolution: The Gateway uses Docker DNS to resolve the hostname movie-service to the dynamic IP address of the running container.
3. Data Retrieval: The Movie Service connects to its isolated SQLite database (movies.db). The service executes a SELECT SQL query to retrieve raw data.

4. **Serialization:** Data from the Database (Rows/Tuples) is serialized into standard JSON format.
5. **Response:** The JSON is sent back through the Gateway to the Frontend for rendering.

B. Distributed Booking Transaction

This is the most complex process, applying the "Choreography" model in the Saga Pattern (simplified) to ensure data consistency.

- **Step 1: Validation & Locking (At Booking Service)**
 - The Service receives a seat booking request.
 - Executes "Check-then-Act": Checks booking.db to see if the seat is already taken.
 - Uses a local SQLite Transaction to ensure Atomicity: If the seat is free, it inserts a Booking record with *PENDING_PAYMENT* status and locks the seat, only switch to *CONFIRMED* after the payment is successfully completed.
 - If this step fails (Database error, duplicate seat), the API returns an error immediately (Fail Fast).
- **Step 2: Event Publication (At Booking Service)**
 - After a successful commit to the Database, the Booking Service acts as a Producer.
 - It creates a Message containing the minimum necessary information (Payload): BookingID, UserID, Amount, ShowtimeInfo.
 - The Message is pushed directly to the RabbitMQ Queue (*ticket_events*).
 - *Note:* The response to the user (HTTP 201 Created) occurs in parallel or immediately after pushing the message, ensuring an extremely fast interface response (Non-blocking I/O).
- **Step 3: Message Routing (At RabbitMQ)**
 - The Booking Service sends messages directly to the *ticket_events* queue. The Payment Service (upon successful transaction) sends messages to the *invoice_events* queue.
- **Step 4: Event Consumption (At Notification & Payment Services)**

- These two services operate independently (Decoupled).
- Notification Service: Consumes messages from multiple queues (**ticket_events** for emails, **invoice_events** for invoices). It uses the payload info to render the HTML ticket template and sends it via the SMTP Server.
- Payment Service: Exposes a REST API to process transactions. Upon success, it publishes an *INVOICE_PRINT* event to the *invoice_events* queue for the Notification Service to consume.
- Acknowledge Mechanism (ACK): After processing is complete, the Service sends an ACK signal to RabbitMQ to delete the message from the queue. If the Service crashes during processing, the message is not lost but is re-queued by RabbitMQ for later processing.

5.4 Distributed Data Management

The system adheres to the Database-per-Service principle:

- Independence: The Movie Service cannot directly access booking.db and vice versa. This prevents "Coupling" at the data layer.
- Data Sharing: When the Booking Service needs movie information (e.g., Movie Title, Showtime) to save on the ticket, it does not join tables (since they are in different databases). Instead, the Frontend sends this information in the payload, or the Booking Service stores a reference ID (*movie_id*) and makes a synchronous API call to the Movie Service if verification is needed.

5.5 Fault Tolerance

This architecture is designed to prevent a total system collapse when a single component fails (Single Point of Failure elimination - excluding the Gateway):

- Scenario: If the Notification Service crashes.
- Consequence: The user still books the ticket successfully. The email is simply not sent immediately.
- Self-Healing: The message remains safe in the RabbitMQ queue. When the Notification Service is restarted (by Docker Compose), it will consume the backlog of messages and send the emails retroactively. This is a critical "Resilience" feature of the system.

6. TESTING & VERIFICATION

6.1 Verification of Requirements (Labs Progress)

The project followed a strict development roadmap, verifying functionality at each stage:

- Functional Verification: All CRUD operations and the full Booking -> Payment->Email flow have been manually verified via the React Frontend.
- Architecture Verification: Successful decomposition into 5 distinct Docker containers, confirming the Microservices boundary analysis.

6.2 Quality Attribute Testing

- Fault Tolerance Test: We simulated a crash of the *Notification Service* while a user was booking.
 - *Result:* The booking was successful (User received HTTP 201). The email was sent automatically once the Notification Service was restarted.
- Performance Test (Debounce): We monitored network traffic during a search operation.
 - *Result:* Without debounce: "Avatar" generated 6 requests. With debounce: "Avatar" generated 1 request.

7. CONCLUSION & REFLECTION

7.1 Summary of Lessons Learned

- Decoupling Complexity: Moving from Monolith to Microservices significantly increased infrastructure complexity (networking, consistency) but provided superior modularity.
- Eventual Consistency: We learned to handle data consistency not through global transactions (ACID), but through Eventual Consistency using Message Queues.

7.2 Future Improvements

- Caching Layer: Implement Redis to cache movie schedules, further reducing load on the Movie Service database.
- CI/CD Pipeline: Integrate GitHub Actions to automatically run unit tests and build Docker images upon code commit.

- Centralized Logging: Implement the ELK Stack (Elasticsearch, Logstash, Kibana) to aggregate logs from all containers for easier debugging.